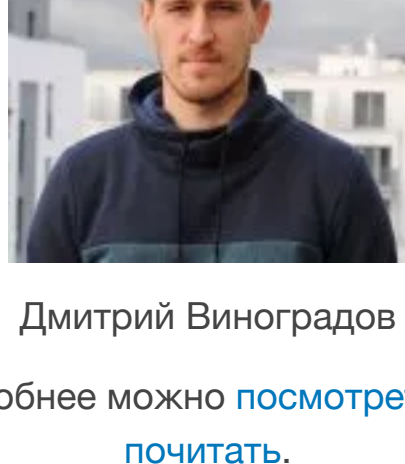


Language



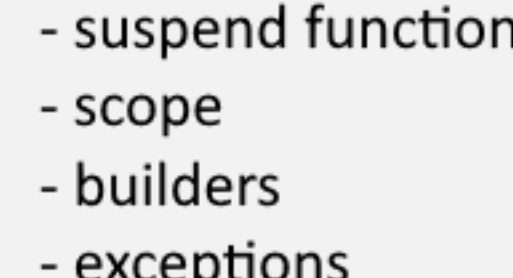
Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

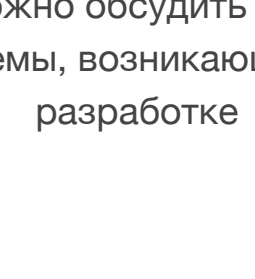
Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



Корутины

- suspend function
- scope
- builders
- exceptions
- flow

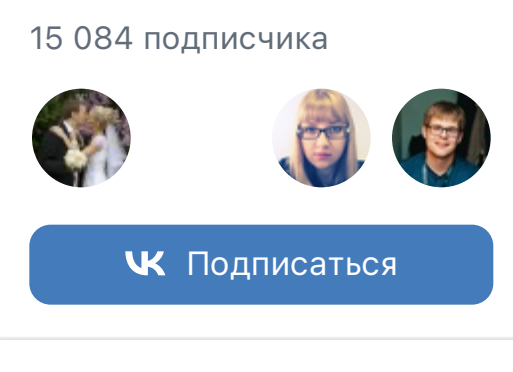
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

Яндекс

410011180491924

WebMoney

R248743991365

Z551306702056

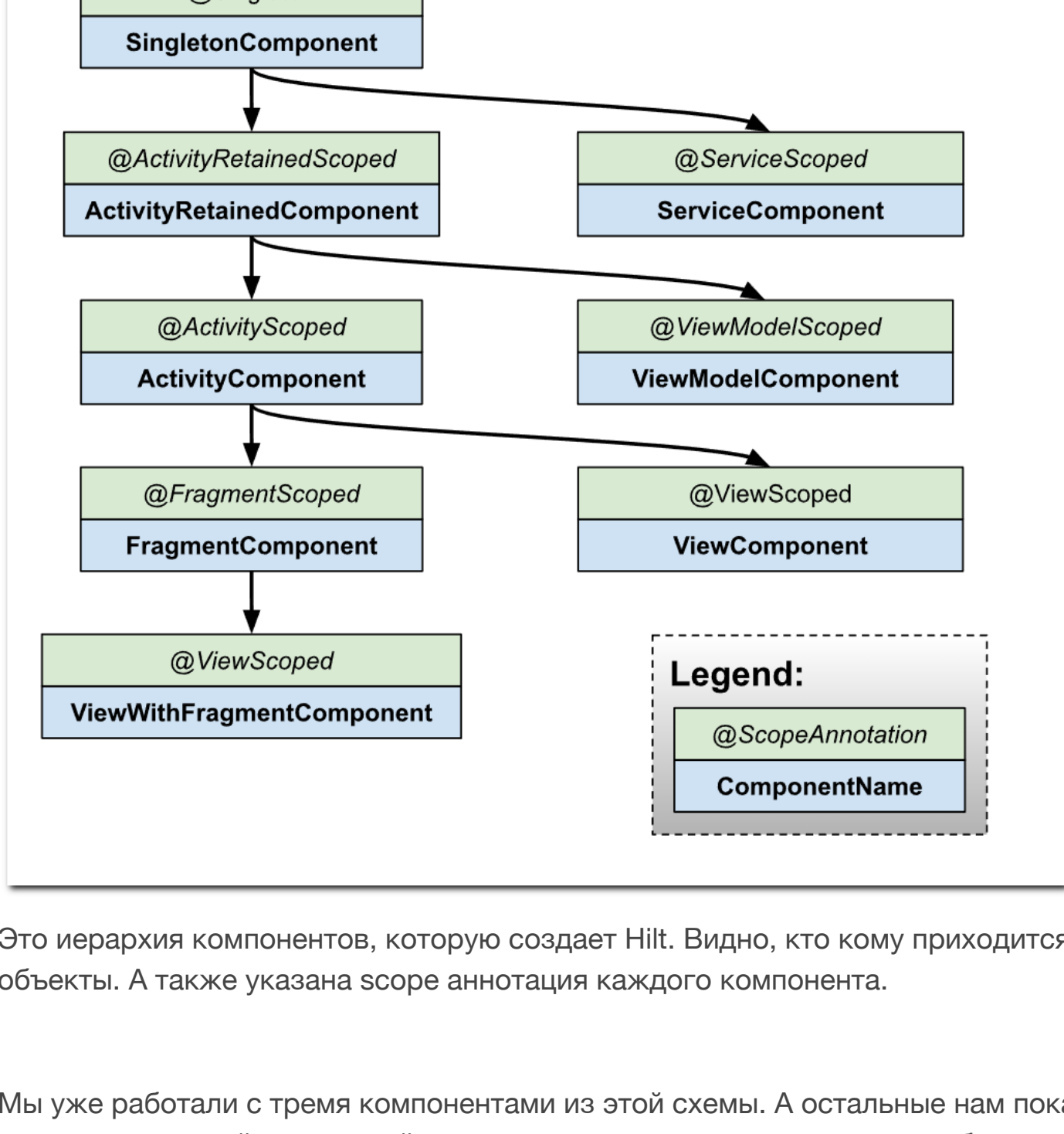
Paypal

Урок 18. Hilt. Компоненты

29 октября 2021

В этом уроке рассмотрим оставшиеся Hilt компоненты

В [официальной документации](#) есть такая схема:



Это иерархия компонентов, которую создаст Hilt. Видно, кто кому приходится субкомпонентом, т.е. кто у кого может брать объекты. А также указана score аннотация каждого компонента.

Мы уже работали с тремя компонентами из этой схемы. А остальные нам пока не знакомы. Давайте по порядку сверху вниз и слева направо пойдем по этой иерархии, и для каждого компонента обсудим следующие пункты:

- для какого Android объекта создается компонент и как долго он живет
- какая Entry аннотация используется
- когда создается компонент и выполняется индекс объектов
- какую Score аннотацию необходимо повесить на объект, чтобы компонент держал синглтон этого объекта
- какие Android объекты сможет предоставить этот компонент в конструкторы создаваемых им объектов

SingletonComponent

Про него мы уже много говорили. Я его обычно называл AppComponent в прошлых уроках.

Этот компонент используется для индекса объектов в App классе. Его время жизни совпадает с временем жизни приложения.

Чтобы Hilt создал компонент, необходимо использовать аннотацию @HiltAndroidApp в App классе:

```
1 @HiltAndroidApp
2 class App: Application() {
3     // ...
4     // ...
5     // ...
6 }
```

Создание компонента и индекс объектов будут выполнены в Application.onCreate.

Score аннотация: @Singleton

Доступные объекты: Application

ActivityRetainedComponent

В прошлых уроках про Hilt я говорил что ActivityComponent является субкомпонентом для AppComponent. На самом деле это не совсем так. Между ними есть еще один субкомпонент.

Т.е. иерархия компонентов выглядит не так: AppComponent > ActivityComponent

а вот так: AppComponent > ActivityRetainedComponent > ActivityComponent

Но это абсолютно ничего не меняет в отношениях между AppComponent и ActivityComponent.

Зачем тогда нужен ActivityRetainedComponent? Это вопрос времени жизни компонента.

ActivityComponent живет, пока живет Activity. При повороте экрана он будет пересоздан. И если он держал какой-то синглтон, то этот синглтон тоже будет пересоздан.

А ActivityRetainedComponent переживает поворот экрана. Он жив, пока мы явно не закроем Activity. И, соответственно, его синглтоны будут жить все это время. И после поворота экрана мы сможем получить тот же объект, что был у нас до поворота. Т.е. поведение очень похоже на ViewModel.

Примечание: сам по себе этот компонент не индексирует объекты в Activity, этим продолжает заниматься ActivityComponent.

ActivityRetainedComponent тут выступает хранилищем синглтонов для ActivityComponent. При поворотах экрана ActivityRetainedComponent может хранить нужные нам синглтоны. А ActivityComponent хоть и будет пересоздаваться с каждым поворотом, но сможет взять синглтон из ActivityRetainedComponent и заиндексировать его в Activity.

Чтобы использовать этот компонент, мы добавляем в Activity обычную аннотацию @AndroidEntryPoint:

```
1 @AndroidEntryPoint
2 class OrderActivity : AppCompatActivity() {
3     // ...
4     // ...
5     // ...
6 }
```

Это приведет к созданию как обычного ActivityComponent, так и ActivityRetainedComponent. И если мы хотим, чтобы синглтоны хранились в ActivityRetainedComponent и могли пережить поворот экрана, то в качестве Score аннотации для объектов используем ActivityRetainedScoped.

Доступные Android объекты, которые этот компонент сможет использовать при создании наших объектов - это только Application.

ServiceComponent

Этот компонент используется для индекса объектов в сервисе. Его время жизни совпадает с временем жизни сервиса.

Чтобы Hilt создал компонент, используем в сервисе стандартную аннотацию @AndroidEntryPoint:

```
1 @AndroidEntryPoint
2 class OrderService : Service() {
3     // ...
4     // ...
5     // ...
6 }
```

Создание компонента и индекс объектов будут выполнены в onCreate.

Score аннотация: @ServiceScoped

Доступные объекты: Application, Service

ActivityComponent

Про этот компонент мы уже все знаем. Кратко резюмируем.

Он используется для индекса объектов в Activity. Его время жизни совпадает с временем жизни Activity.

Для создания используем стандартную аннотацию @AndroidEntryPoint:

```
1 @AndroidEntryPoint
2 class OrderActivity : AppCompatActivity() {
3     // ...
4     // ...
5     // ...
6 }
```

Создание компонента и индекс объектов будут выполнены в onCreate.

Score аннотация: @ActivityScoped

Доступные объекты: Application, Activity

ViewModelComponent

Используется для индекса объектов в ViewModel. Время жизни совпадает с временем жизни модели.

Чтобы Hilt создал компонент, используем в модели аннотацию @HiltViewModel:

```
1 @HiltViewModel
2 class OrderViewModel @Inject constructor(...): ViewModel() {
3     // ...
4     // ...
5     // ...
6 }
```

В конструкторе модели просто указываем все объекты, которые нам нужны. Далее мы обычным путем (через провайдер) запрашиваем эту модель у фрагмента или Activity, и получаем ее, уже наполненную объектами.

Score аннотация: @ViewModelScoped

Доступные объекты: Application, SavedStateHandle

Обратите внимание, что родителем для этого компонента является ActivityRetainedComponent. Т.е. ViewModel имеет доступ к его синглтонам. И тут может возникнуть вопрос: чем синглтоны ViewModelComponent отличаются от синглтонов ActivityRetainedComponent.

Для каждой ViewModel создается свой экземпляр ViewModelComponent. А, значит, у каждой ViewModel будет свой синглтон объекта с аннотацией @ViewModelScoped.

А вот если мы на объект вешаем аннотацию @ActivityRetainedScoped, то такой синглтон будет общим для всех моделей текущего Activity.

FragmentComponent

Про этот компонент мы тоже уже знаем. Кратко резюмируем.

Используется для индекса объектов во фрагмент. Его время жизни совпадает с временем жизни фрагмента.

Чтобы Hilt создал компонент, используем во фрагменте стандартную аннотацию @AndroidEntryPoint:

```
1 @AndroidEntryPoint
2 class OrderFragment : Fragment() {
3     // ...
4     // ...
5     // ...
6 }
```

Создание компонента и индекс объектов будут выполнены в onAttach.

Score аннотация: @FragmentScoped

Доступные объекты: Application, Activity, Fragment

ViewComponent

Используется для индекса объектов в View. Его время жизни совпадает с временем жизни View.

Чтобы Hilt создал компонент, используем в View стандартную аннотацию @AndroidEntryPoint:

```
1 @AndroidEntryPoint
2 class OrderView(context: Context, attrs: AttributeSet): AppCompatActivity(context, attrs) {
3     // ...
4     // ...
5     // ...
6 }
```

Создание компонента и индекс объектов будут выполнены в вызове super().

Score аннотация: @ViewScoped

Доступные объекты: Application, Activity, View

ViewWithFragmentComponent

Аналогичен предыдущему компоненту, но в качестве родителя использует FragmentComponent, а не ActivityComponent. Это может быть полезным, если есть необходимость доступа к объектам FragmentComponent

К стандартной аннотации @AndroidEntryPoint необходимо добавить аннотацию @WithFragmentBindings

```
1 @WithFragmentBindings
2 @AndroidEntryPoint
3 class OrderView(context: Context, attrs: AttributeSet): AppCompatActivity(context, attrs) {
4     // ...
5     // ...
6     // ...
7 }
```

Доступные объекты: Application, Activity, Fragment, View

Вывод

Во всем этом многообразии компонентов и score важно понять следующее.

Добавляя к своему Android объекту аннотацию @AndroidEntryPoint (и т.п.), мы создаем для него компонент. Этот компонент теперь будет индексировать нам нужные объекты.

Кроме обычных объектов, компонент умеет индексировать Android объекты.

Если какой-либо объект мы хотим получить, как синглтон, то мы вешаем на него score аннотацию. Выбирая для объекта score аннотацию, мы определяем время жизни этого объекта.

Например, если мы в View хотим от его компонента ViewComponent получить объект, который живет все время, пока живет Activity, то такой объект мы пометим аннотацией ActivityScoped. Этот объект будет создан в ActivityComponent, и уже оттуда его возьмет ViewComponent и заиндексирует в View.

Назад

Вперед

Комментарии

RE: Урок 18. Hilt. Компоненты — Vlad_Lav 05.11.2021 09:39

Ответить

RE: Урок 18. Hilt. Компоненты — Artem 28.11.2021 18:42

Отличный урок, в который раз спасибо!

Ответить

Зачем ViewModel с DI? — Виктор 11.02.2022 18:14

"ActivityRetainedComponent переживает поворот ... Т.е. поведение очень похоже на ViewModel."

При использовании DI не понятно зачем вообще применять ViewModel? Почему не инжектировать сразу в Activity? Фрагменты всё же нужно. Полезного во VM остается только onCleared() когда необходимо что-то заархивировать?

Ответить

RE: Зачем ViewModel с DI? — Dmitry Vinogradov 18.02.2022 12:20

Ну для Activity, возможно. Там можно использовать ActivityRetained, чтобы получить похожее на ViewModel поведение. Да и то будет отличие, которое я упомянул в описании ViewModelComponent.

А для фрагмента будет сложновато создать компонент, который будет держать синглтоны только на время жизни этого фрагмента.

Ответить

почему сложно? — Даниил 28.11.2023 10:41

Вот же вы писали: "FragmentComponent"

Про этот компонент мы тоже уже знаем. Кратко резюмируем.

Используется для индекса объектов во фрагмент. Его время жизни совпадает с временем жизни фрагмента."

Ответить

RE: почему сложно? — Dmitry Vinogradov 13.12.2023 20:48

Я имел ввиду синглтоны, которые переживают поворот экрана.

Ответить

Обновить список комментариев

RSS лента комментариев этой записи

Добавить комментарий

Тема

👍👎😄😁😂😃😅😆😇😈😉😊😋😌😍😎😏😐😑😒😓😔😕😖😗😘😙😚😛😜😝😞😟😠😡😢😣😤😥😦😧😨😩😪😫😬😭😮😯😰😱😲😳😴😵😶😷😸😹😺😻😼😽😾😿😺

Отправить

Comments

© StartAndroid, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте [форму обратной связи](#)

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under [MIT License](#).

Font Awesome font licensed under [SIL OFL 1.1](#).